

Polite Bursting: What Feedback Is Worth When Admissions Land Late on a Shared Cluster

Abstract—Research teams run batch GPU work on shared clusters they do not own. Fair-use policy requires each pod a guest holds to stay busy and scrutinizes guests that hold many at once, so a guest has to restrain itself. How much capacity a guest can actually use is set by the other tenants. It is not announced, and a guest learns it only by probing. Worse, a decision acts on information from before the probe and takes effect only after a new worker starts, so the state it is judged against has moved on by the time it lands. We show that acting on that lag has an exact ceiling. For a contended resource that is either free or taken, the extra goodput any feedback controller can win over the best fixed budget, compared at equal over-admission, equals half the correlation between capacity when the decision lands and capacity as it was last seen. A rule that admits work whenever the last reading showed the resource free attains that ceiling. The ceiling falls toward zero once the lag grows past the time over which neighbour load stays correlated, so past that point feedback is worth little however it is built. We give the elementary derivation rather than claim it, since the ingredients are classical. We turn the result into one controller. It estimates the correlation from its own probe stream and switches between a feedback loop and a fixed budget, and we bound how fast any rule can learn which of the two to run. That learning cost grows with the square root of the lag even though the ceiling it chases decays exponentially in the same quantity, and the hardest predictability level to sit at is an intermediate one rather than either extreme. Short controlled experiments measure the lag at about 2.5 seconds on a two-GPU node, dominated by worker start-up, and exercise the controller against a competing workload we drive ourselves. Under structured neighbour load feedback admission delivers 1.41 times the goodput of a fixed trickle at a quarter of a greedy policy’s over-admission while leaving the competitor more of its own throughput. Under memoryless load it only breaks even, which the ceiling predicts, and the switch detects that case and stops paying for the loop. The live comparison is at one operating point and so measures the system rather than the constant.

I. INTRODUCTION

Much machine-learning work is bursty. Long stretches of local development are punctuated by short runs of batch jobs, sweeps, fine-tunes and evaluations, that briefly need far more GPUs than a team owns. Shared clusters such as the National Research Platform [1] make that capacity available on guest terms. Fair-use policy caps concurrent pods per user, requires sustained utilization of what is requested, forbids idle or sleep-only work, and reclaims resources on completion. The guest also sits outside the cluster network, often reachable only through one forwarded port or a mesh VPN, with no inbound access.

Two obvious strategies both fail here. Submitting everything at once takes more than a fair share, leaves pods pending or idle, and trips the thresholds that get a namespace banned.

A fixed submission rate is safe but wastes the guest’s own throughput whenever capacity is free. Neither one reacts to what the cluster is doing.

Being a good guest is a control problem, and how well it can be solved depends on how stale the guest’s view of the cluster is. This paper makes that dependence exact and then builds admission algorithms on it.

Our first result is a ceiling on the value of feedback. When a controller acts on a capacity reading of a given age, the extra goodput it can win over the best fixed budget equals half the correlation between capacity now and capacity as last seen. The ceiling is tight, since a rule that admits work whenever the last reading showed capacity free reaches it. It also implies an impossibility. Once the sensing delay exceeds the time over which neighbour load stays correlated, the correlation is near zero and no feedback controller can beat a fixed budget.

Our second result turns the ceiling into a controller. The correlation it needs is measurable from the probe stream it already collects, so a guest can estimate it online and act on the estimate. Algorithm 1 does this with a handful of counters, switching between a feedback loop and a fixed budget, and on average it trails the better of the two by an amount that shrinks with run length. We bound the cost of that decision from both sides. We give matching upper and lower bounds on how fast any rule can learn which branch to take, and together they show that the lag costs only its square root in learning even though it costs exponentially in the ceiling itself. The hardest predictability level to sit at is neither high nor low but a specific value in between, which is exactly where a controller holding no state has no good default. Integer capacity beyond one contended unit is left open, for reasons §V states.

Our third contribution is measurement rather than proof. The sensing delay is a physical quantity and we measure it at roughly 2.5 seconds on real hardware, dominated by CUDA start-up rather than by polling. We then drive a two-GPU node with a competing workload of our own construction and show the predicted behaviour on both sides of the crossover, including a case where feedback stops paying for itself and the switching algorithm gives it up.

The control plane runs inside the cluster and reaches the home node over a single outbound port. The fabric that carries it is characterized in a companion paper [2] and is treated here as given, since what matters for the result is only that the guest’s view of capacity arrives late.

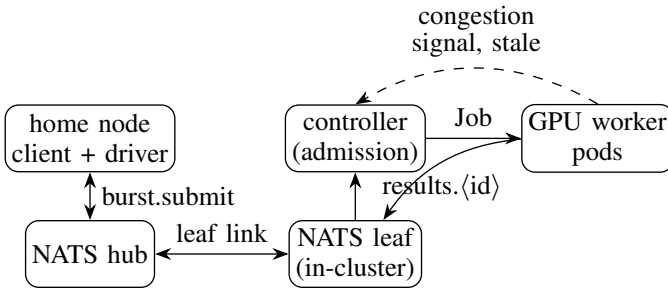


Fig. 1. PoliteBurst architecture. The home client submits over a federated NATS leaf link. The in-cluster controller admits jobs, pacing concurrency to the politeness budget, and launches GPU worker pods whose results route back through the leaf. The dashed arc is the stale feedback the controller acts on, and its delay bounds the achievable advantage by Thm. 1.

II. BACKGROUND AND MOTIVATION

Host policy. The reference host [1] publishes a fair-use policy with two features our design depends on. Utilization relative to request must stay within published bands for each running pod, and holding many concurrent pods brings a namespace under closer scrutiny for exactly that condition. Jobs must do real work, since sleep-only jobs are banned, and must release resources on completion. Administrators monitor utilization and act on namespaces that persistently hold under-utilized requests, so restraint is not optional.

We therefore do *not* model the policy as an absolute pod maximum. The published rules attach utilization conditions to concurrency rather than forbidding it outright, and the permitted concurrency depends on resource type and node. Instead K is a *self-imposed politeness window* that the guest sets at or below the concurrency at which those conditions begin to apply, and ϵ is the share of under-utilized epochs the guest is willing to risk. This is the weaker and more defensible reading, and nothing below depends on the particular value. The experiments use $K=4$ and a contended $C=2$ node.

Network reality. The guest reaches the cluster over one forwarded port or a mesh VPN, with no inbound access. A control plane must therefore live behind that boundary and communicate over a fabric that traverses it, which motivates the leaf-node design of §VI.

Why existing tools do not settle it. Workflow and function-execution frameworks such as HTCondor [3], funcX and Globus Compute [4] and Ray [5] scale workers to demand and carry no notion of a host politeness budget. Left to default scaling they over-provision under contention. We make self-restraint a controlled objective instead.

III. SYSTEM MODEL

A guest bursts batch GPU pods onto a shared cluster under a fair-use policy that allows at most K concurrent pods and requires each admitted pod to sustain a utilization floor. Time is slotted into control epochs $t \in \mathbb{N}$. Let $a_t \in \{0, \dots, C\}$ be the capacity the cluster can productively give the guest at epoch t . It is set by the other tenants, so it is exogenous, stochastic and not directly observable. The

guest chooses a window $w_t \in \{0, \dots, K\}$. Useful goodput is $G_t = \min(w_t, a_t)$ and the surplus $(w_t - a_t)^+$ is over-admitted. We are deliberately careful about what that surplus costs, because the obvious reading is wrong. A pod that requests a GPU when none is allocatable does not run at low utilization, it stays pending, so surplus admissions are not per-pod utilization violations. Their cost is that a pending claim captures capacity the moment the neighbour releases it, and that a guest holding many such claims is what fair-use review looks at. We therefore treat ρ as a measure of *claims in excess of available capacity* and validate it against the quantity it is supposed to proxy, namely the throughput the co-tenant keeps (§VIII-B). The tolerance ϵ is a guest-chosen budget on that excess rather than a figure read off the published policy. Writing $\rho \triangleq \limsup_T \frac{1}{T} \sum_{t < T} \mathbf{1}[w_t > a_t]$ for the long-run over-admission fraction, the guest solves

$$\max_{\{w_t\}} \mathbb{E}[\min(w_t, a_t)] \quad \text{s.t.} \quad \underbrace{w_t \leq K \forall t}_{\text{hard cap}}, \quad \underbrace{\rho \leq \epsilon}_{\text{policy}}. \quad (1)$$

The constraint is two-sided, since the guest must stay below the cap and above the floor, which already separates the problem from loss-only congestion control.

The information-to-effect lag. Two distinct delays act on this loop and only their sum matters. A decision taken at wall-clock time τ rests on a capacity reading of age D_{obs} , and it takes effect at $\tau + D_{\text{act}}$, when the admitted pod actually occupies the resource and its productivity is scored. The state that determines the payoff is therefore the capacity at $\tau + D_{\text{act}}$, while the information the policy conditions on is the capacity at $\tau - D_{\text{obs}}$. Writing

$$D \triangleq D_{\text{obs}} + D_{\text{act}} \quad (2)$$

for that information-to-effect lag and relabelling $t = \tau + D_{\text{act}}$, the admissible policies are exactly the causal functions of $\{a_{s-D}\}_{s \leq t}$. This is the only sense in which D is used below, and it is the sense the measurements report. It is not the age of the record at the decision instant, which is only D_{obs} .

The distinction matters because the two components differ by two orders of magnitude. Polling occupancy of resources already running costs $D_{\text{obs}} = 27\text{--}33$ ms, well under one epoch, so a paper that identified D with observation age alone would predict that feedback is always useful here. Actuation is what dominates. A newly admitted pod needs $D_{\text{act}} \approx 2.46$ s of CUDA start-up before it occupies the GPU it was admitted for (§VIII), and the neighbour keeps moving during that interval. So D is 2 to 3 epochs, not a fraction of one, and it is set by actuation rather than by monitoring design.

That also answers the natural objection that D is an engineering choice. Distributed schedulers routinely hide actuation latency by launching more work than they need and binding to whichever reservation lands first, discarding the rest [6]. Under a one-sided constraint that works. Under the two-sided constraint of (1) it does not.

Remark 1 (Why the actuation term is hard to remove here). For the controller studied here, where a scale-out decision

creates a new CUDA worker, the measured information-to-effect lag is about 2.46 s (§VIII-A) and is dominated by worker start-up. The usual way to hide that latency is to keep spare admitted capacity, whether by speculative replicas that are later cancelled or by an idle standby, and both consume claims against the guest's own window while contributing nothing, which is what the guest is trying to avoid. We do *not* claim this is irreducible in general. A persistent worker pulling from a queue, pipelined replacement that launches a successor while its predecessor still works, or a warm pool all reduce or remove the start-up term, at the cost of holding capacity. Which trade is preferable is a design question, and the results below take D as measured for this design rather than as a universal floor.

What separates this setting from load balancing on stale state, where the classical prescription is to sample few and avoid acting greedily on old readings [7], [8], is that there a guest which does nothing is compliant, so holding spare capacity to hide latency is free. Here it is not. Everything that follows treats D as measured and asks what is achievable at that lag.

The contended unit. Politeness is decided at the margin, so we fix attention on the single contended resource unit whose availability fluctuates with the neighbour. Let $x_t \in \{0, 1\}$ indicate that this unit is free ($x_t = 1$) or taken ($x_t = 0$), so admitting it when taken is the over-admission the policy punishes. We take x_t stationary with $\Pr[x_t = 1] = \frac{1}{2}$, a fully contended unit, and note that the asymmetric case only rescales the constants below. The guest's marginal decision $m_t \in \{0, 1\}$ is a possibly randomized causal function of the age- D observation $\hat{x}_t \triangleq x_{t-D}$. Marginal goodput and over-admission are

$$g \triangleq \Pr[m_t = 1, x_t = 1], \quad \rho \triangleq \Pr[m_t = 1, x_t = 0]. \quad (3)$$

Everything below concerns the achievable (ρ, g) region under lag D . The stationary ρ of (3) is the ergodic limit of the epoch fraction in (1), so we reuse the symbol.

Proposition 1 (Hard-cap safety). *Any window update that clamps additive increase at K and never increases w_t on a congestion signal keeps $w_t \leq K$ for every t deterministically, so the pod cap is never violated.*

Proof. Induction on t . Increase is clamped to K and decrease cannot exceed the current value. $\square$

IV. HOW MUCH STALE FEEDBACK IS WORTH

We now characterize exactly how much closed-loop admission gains over a fixed budget when the only signal is a D -old observation. The answer is one interpretable quantity, the autocorrelation of capacity at the sensing lag.

Lemma 1 (Predictive disagreement). *Let $q(D) \triangleq \Pr[x_t \neq x_{t-D}]$ be the lag- D disagreement probability and $r(D) \triangleq \mathbb{E}[(2x_t - 1)(2x_{t-D} - 1)]$ the lag- D autocorrelation of centered availability. Then $r(D) = 1 - 2q(D) \in [-1, 1]$, and by stationarity and symmetry the joint law of $(\hat{x}_t, x_t)$ is*

$\Pr[\hat{x}_t = i, x_t = j] = \frac{1}{4}(1 + (2i - 1)(2j - 1)r(D))$. *For a symmetric two-state Markov capacity with per-epoch flip probability $p \in (0, \frac{1}{2})$ we have $r(D) = (1 - 2p)^D = e^{-D/\tau_c}$ with coherence time $\tau_c \triangleq -1/\ln(1 - 2p) \approx 1/(2p)$.*

Proof. $r = \Pr[\text{agree}] - \Pr[\text{disagree}] = 1 - 2q$. The joint law follows from the two symmetric marginals and the single correlation. For the Markov chain, writing $\lambda_2 \triangleq 1 - 2p$ for the second eigenvalue, the D -step transition is $P^D = \frac{1}{2} \begin{pmatrix} 1 + \lambda_2^D & 1 - \lambda_2^D \\ 1 - \lambda_2^D & 1 + \lambda_2^D \end{pmatrix}$, so $\Pr[\text{disagree}] = \frac{1}{2}(1 - \lambda_2^D)$ and $r = \lambda_2^D$. The two-state chain's autocorrelation and its total-variation distance to stationarity are standard mixing facts [9]. $\square$

Theorem 1 (Achievable region and the feedback advantage). *For the symmetric two-state Markov capacity of Lemma 1, under age- D sensing the achievable (ρ, g) region of (3) is the quadrilateral with vertices $\{(0, 0), (\frac{q}{2}, \frac{1-q}{2}), (\frac{1}{2}, \frac{1}{2}), (\frac{1-q}{2}, \frac{q}{2})\}$ with $q = q(D)$, whose efficient upper boundary is the polyline $(0, 0) \rightarrow (\frac{q}{2}, \frac{1-q}{2}) \rightarrow (\frac{1}{2}, \frac{1}{2})$. The best observation-oblivious policy is confined to the diagonal $g = \rho$. Consequently the goodput gain of feedback over a fixed budget, maximized over matched over-admission levels, is*

$$\Delta(D) = \frac{1}{2}r(D) = \frac{1}{2}(1 - 2q(D)) \quad (4)$$

attained at the kink $\rho = q/2$ by the threshold policy $m_t = \hat{x}_t$, while at a lower level $\rho < q/2$ the gain is the smaller $\rho(1 - 2q)/q$. The peak value of closed-loop admission is therefore exactly half the autocorrelation of available capacity at the sensing lag (Fig. 2). Optimality of memoryless threshold policies for symmetric two-state Markov observation is classical in opportunistic access [10], and what (4) adds is the exact value of the resulting advantage over the best fixed budget.

Proof. For the symmetric two-state Markov chain the freshest sample $\hat{x}_t = x_{t-D}$ is a sufficient statistic for x_t given the whole delayed history $\{x_{s-D}\}_{s \leq t}$, since the Markov property makes older samples conditionally uninformative. An optimal causal age- D policy therefore depends on the history only through $\hat{x}_t$, so we may restrict without loss to policies parameterized by a pair (θ_1, θ_0) with $\theta_i = \Pr[m_t = 1 \mid \hat{x}_t = i]$. Using Lemma 1, $g = \frac{1}{2}(\theta_1(1 - q) + \theta_0 q)$ and $\rho = \frac{1}{2}(\theta_1 q + \theta_0(1 - q))$. This linear map sends the square $[0, 1]^2$ to the quadrilateral with the stated vertices, whose upper boundary is the stated hull. Maximizing $g - \lambda\rho$ turns on θ_1 before θ_0 because $q < \frac{1}{2}$ makes the free-observation state strictly more goodput-efficient, so the maximum-goodput point for $\rho \leq q/2$ is the kink $(\frac{q}{2}, \frac{1-q}{2})$, achieved by $m_t = \hat{x}_t$. A fixed policy ignores $\hat{x}_t$, so $\theta_1 = \theta_0 = \theta$ and $g = \rho = \theta/2$, the diagonal. The vertical distance at $\rho = q/2$ is $\frac{1-q}{2} - \frac{q}{2} = \frac{1-2q}{2} = \frac{1}{2}r(D)$. $\square$

Corollary 1 (The advantage decays with the lag). $\Delta(D) \rightarrow 0$ as $r(D) \rightarrow 0$. *For the symmetric two-state Markov model $\Delta(D) = \frac{1}{2}e^{-D/\tau_c}$, so any causal controller at lag D gains at most $\frac{1}{2}e^{-D/\tau_c}$ over a fixed budget, which is below any tolerance η once $D \geq \tau_c \ln(1/2\eta)$. Closed-loop admission*

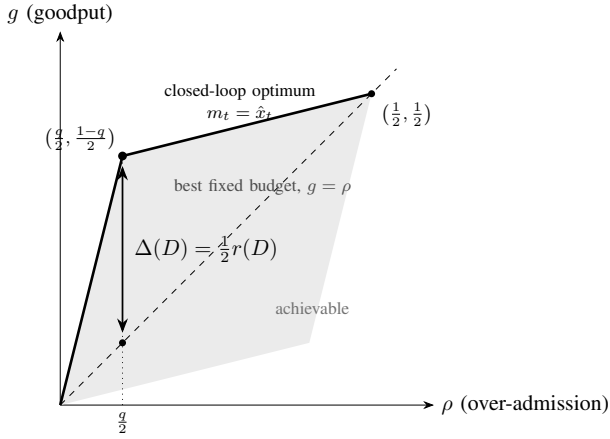


Fig. 2. Under lag- D sensing the achievable (ρ, g) pairs form the shaded hull, drawn at $q = 0.2$. Fixed budgets lie on the diagonal $g = \rho$, and the closed-loop optimum sits a vertical $\Delta(D) = \frac{1}{2}r(D)$ above it (Thm. 1). The advantage is a distance, which is why it admits a closed form.

helps only to the extent that the guest senses the neighbour faster than the neighbour changes. In this model low auto-correlation and unpredictability coincide, but in general they need not, since a decorrelated process with $r(D)=0$ can still be exactly predictable from its history, as a random-phase deterministic square wave is. For an arbitrary capacity process the vanishing of the advantage is therefore governed by the β -mixing coefficient rather than by $r(D)$. At $D = 0$ we have $\Delta = \frac{1}{2}$, and the advantage halves for every $\tau_c \ln 2$ epochs of added age (Fig. 2). App. A makes the general statement precise and extends the impossibility to arbitrary non-binary and non-Markov capacity processes.

Provenance, stated plainly. Equation (4) is elementary and we would rather derive it twice than appear to have hidden how short it is. For a symmetric binary hypothesis the Bayes error is $\frac{1}{2}(1 - \|\cdot\|_{TV})$, so the largest advantage any test holds over blind guessing is exactly half the total variation between the two conditionals [11]. For a two-state chain that total variation is $|\lambda_2|^D$, which is $r(D)$. Compose the two and (4) follows in two lines. Each ingredient is old. Value of information is non-negative and convex in the prior [12] and is bounded by payoff oscillation times the movement of beliefs [13]. That movement at age D is the β -mixing coefficient [14], and the mixing facts for a two-state chain are standard [9]. Threshold optimality for positively correlated two-state availability is Ahmad et al. [10], and threshold optimality under state-information lag is Kim [15]. Measuring the value of information as degradation from the perfect-information gain is Brooks and Leondes [16]. Reducing a delay- D decision problem to an undelayed one by state augmentation is Katsikopoulos and Engelbrecht [17].

So the optimal policy is not ours and neither is the formula. Three things are. First, the statement is a *converse over every causal controller at matched over-admission*, not the value of one policy. The matched-constraint normalization is what makes “advantage over the best fixed budget” well posed at

all, and it is the actual technical content of Thm. 1. Second, App. A lifts the converse off the binary Markov case to any stationary capacity process. Third, and most important for a systems venue, Rem. 1 records that the lag in this design is dominated by a term the guest cannot remove without holding spare capacity, which is what makes a lag-indexed limit the useful object here rather than a modelling convenience. The exogenous structure also matters. Unlike the driven-plant setting of networked control, the admission action does not affect capacity, and that is what collapses the value of feedback onto a single predictability coefficient.

Corollary 2 (The two contention profiles are two points on (4)). *The contention study of §VIII instantiates this two-state $C=2$ model. Structured square load has $\tau_c \gg D$, so $r(D)$ is near 1 and feedback is near-maximally useful. Memoryless Poisson load drives $\tau_c \lesssim D$, so $r(D)$ is near 0 and feedback collapses onto the fixed budget, which is the pre-registered null. The single curve (4) predicts both cases and the crossover between them.*

V. AN ADAPTIVE CONTROLLER

Theorem 1 leaves a shortfall that no fixed controller closes. Additive-increase and multiplicative-decrease tracks a slowly varying neighbour and approaches the kink when $\tau_c \gg D$, but over-admits when $\tau_c \lesssim D$, since it keeps probing a signal that has become noise. A fixed budget is safe everywhere and forfeits $\Delta(D)$ when $\tau_c \gg D$. Which of the two is right depends on a quantity the controller can measure, because $r(D)$ is an autocorrelation of the probe stream the controller already collects.

Algorithm 1 estimates it online and switches. Beyond the AIMD window it holds a flip count, a sample count, the previous reading and a D -deep ring of past windows, so its state is $O(D)$ and its per-epoch cost is constant. The window w is a real-valued target and concurrency is $\lfloor w \rfloor$, which matters because $\alpha < 1$ otherwise leaves the window unable to move. It inherits Prop. 1 and therefore never exceeds the pod cap. The counters accumulate over the whole run rather than over a sliding window. That matters for Lemma 2, since a fixed window of length n leaves a floor on the misclassification probability and the shortfall would then not vanish. A sliding window is the right choice under drift and is what §VIII uses, at the cost of the asymptotic statement.

The tracking analysis of Algorithm 1 assumes **(A1)** stationary capacity $a_t \equiv a$ with $w_{\min} \leq a \leq K$, **(A2)** reliable detection of an overshoot within D epochs, and **(A3)** fixed $\alpha, \beta, w_{\min}, K$.

Proposition 2 (Sawtooth realization of the kink). *Under stationary capacity a , assumptions (A1) to (A3), a backoff deep enough to clear the overshoot so that $\beta(a + D\alpha) \leq a$, and ignoring the integer rounding of the window, the trackable branch of Algorithm 1 forms a periodic sawtooth about a with long-run over-admission $\rho = D\alpha / ((1 - \beta)(a + D\alpha)) \approx D\alpha / ((1 - \beta)a)$. Choosing $\alpha \leq \epsilon(1 - \beta)a/D$ gives $\rho \leq \epsilon$, and $\rho \rightarrow 0$ as $D \rightarrow 0$.*

Algorithm 1 Adaptive admission for one contended unit

Require: cap K , floor $w_{\min}$, step α , backoff β , switch level θ , lag D , fixed budget $\bar{a}$

```

1:  $w \leftarrow w_{\min}; F \leftarrow 0; n \leftarrow 0; \hat{a}_{\text{prev}} \leftarrow \perp$ 
2: for each control epoch  $t$  do
3:    $\hat{a} \leftarrow$  newest capacity reading  $\triangleright$  age  $D_{\text{obs}}$ ; a decision
   now lands at lag  $D$ 
4:   if  $\hat{a}_{\text{prev}} \neq \perp$  then
5:      $F \leftarrow F + \mathbf{1}[\hat{a} \neq \hat{a}_{\text{prev}}]; n \leftarrow n + 1$ 
6:   end if
7:    $\hat{a}_{\text{prev}} \leftarrow \hat{a}$ 
8:    $\hat{p} \leftarrow \min(F / \max(n, 1), \frac{1}{2}) \triangleright$  clamp, else  $(1 - 2\hat{p})^D$ 
   turns positive again for even  $D$ 
9:    $\hat{r} \leftarrow (1 - 2\hat{p})^D$ 
10:  if  $\hat{r} > \theta$  then  $\triangleright$  trackable, so run the loop
11:     $w_{\text{eff}} \leftarrow$  window admitted  $D$  epochs ago  $\triangleright$  from a
     $D$ -deep ring of past windows
12:    if  $w_{\text{eff}} > \hat{a}$  then  $\triangleright$  that admission over-admitted
13:       $w \leftarrow \max(w_{\min}, \lceil \beta w \rceil)$ 
14:    else
15:       $w \leftarrow \min(w + \alpha, K)$ 
16:    end if
17:  else  $\triangleright$  not trackable, so stop paying for the loop
18:     $w \leftarrow \min(\lfloor \bar{a} \rfloor, K)$ 
19:  end if
20:  push  $w$  onto the ring; admit  $\lfloor w \rfloor$  pods  $\triangleright w$  is a
  real-valued target, concurrency is its floor
21: end for

```

Proof sketch. After a backoff the window restarts at $\beta(a + D\alpha)$ and rises by α per clean epoch, overshooting a for D epochs before detection triggers the next backoff at peak $a + D\alpha$. The fraction of violating epochs per cycle gives ρ . Rounding the window to integers perturbs the cycle length by at most one epoch and the bound by $O(\alpha/a)$. This is the dynamic approach to the static-capacity kink of Thm. 1, and D enters both here and in (4), which is the hinge between the systems measurement and the theory. $\square$

Learning which branch to take has a sharp rate, and it is not the rate a generic chain concentration bound suggests. For the symmetric two-state chain the flip indicators $\mathbf{1}[a_t \neq a_{t-1}]$ are *exactly* independent Bernoulli(p) variables, since the flip probability is p from either state and after every history. Estimating p therefore carries no mixing penalty, and the relevant variance is $p(1-p)$ rather than the $1/\gamma$ proxy that a reversible-chain Hoeffding bound inserts. Since $\gamma = 2 \min(p, 1-p)$, that true variance is about $\gamma/2$, so the spectral gap belongs in the numerator. We therefore use the Bernstein form of [18], which is stated in the asymptotic variance and is tight here.

Write p_θ for the solution of $(1 - 2p)^D = \theta$, let $s_\theta \triangleq 2D\theta^{(D-1)/D}$ be the sensitivity $|dr/dp|$ at that point, and let $\sigma_\theta^2 \triangleq p_\theta(1 - p_\theta)$. Let $L > 0$ be the local slope in r of the difference between the two branches' values at θ , which is positive because Prop. 2 and Cor. 1 cross transversally there.

What can be said sharply is how hard the *decision* is, separately from what either branch is worth. Choosing a branch means deciding on which side of θ the quantity $r(D)$ lies, and that is a statistical problem with a rate. We state it as such, and are explicit below about the step we do not take.

Lemma 2 (Branch-decision rate). *Consider deciding, from N consecutive age- D readings of a symmetric two-state capacity process near p_θ , whether $r(D) > \theta$. The local minimax error in r of any causal rule is*

$$\Theta\left(\frac{s_\theta \sigma_\theta}{\sqrt{N}}\right), \quad s_\theta \sigma_\theta = D\theta^{\frac{D-1}{D}} \sqrt{1 - \theta^{2/D}}, \quad (5)$$

and the estimator inside Algorithm 1 attains it. Maximizing over the switch level,

$$\max_{\theta \in (0,1)} s_\theta \sigma_\theta = \left(1 - \frac{1}{D}\right)^{\frac{D-1}{2}} \sqrt{D} \rightarrow e^{-1/2} \sqrt{D}, \quad (6)$$

attained at $\theta = ((D-1)/D)^{D/2}$, which increases to $e^{-1/2}$. When $r(D)$ is bounded away from θ the error decays as $O(1/N)$ provided the counters accumulate over the run.

Proof sketch. Upper bound. Bernstein for the independent flip indicators [18] gives $|\hat{p}_n - p| = O(\sigma_\theta/\sqrt{n})$ with high probability, and the delta method through $p \mapsto (1 - 2p)^D$, whose derivative at p_θ is s_θ , gives $|\hat{r}_n - r(D)| = O(s_\theta \sigma_\theta/\sqrt{n})$. Lower bound. Take two processes whose flip probabilities differ by δ . Their per-sample Kullback-Leibler divergence is $\delta^2/(2\sigma_\theta^2)$, so no rule separates them over N samples unless $\delta \gtrsim \sigma_\theta/\sqrt{N}$, which through the same map means correlations closer than $s_\theta \sigma_\theta/\sqrt{N}$ cannot be distinguished. Le Cam's two-point argument [11] gives (5), and the two processes differ in flip probability by $O(N^{-1/2})$ so their spectral gaps agree to that order. For (6), substituting $u = 1 - 2p$ gives $s_\theta \sigma_\theta = Du^{D-1} \sqrt{1 - u^2}$, maximized at $u^2 = (D-1)/D$ where $\sqrt{1 - u^2} = 1/\sqrt{D}$, and $\theta = u^D$. $\square$

What this gives, and what it does not. Lemma 2 concerns the decision, not goodput. Converting it into a goodput shortfall needs the value of each branch as a function of p , and we do not derive those. Prop. 2 gives the AIMD sawtooth under *constant* capacity rather than an AIMD value curve under a Markov neighbour, so we have no proof that the two branch values cross transversally at any particular θ , and we therefore claim no minimax optimality for Algorithm 1 itself. What the lemma does fix is the shape of the learning cost. The spectral gap cancels, since the flip indicators are independent and carry no mixing penalty, so a rate written as $1/\sqrt{\gamma_0 N}$ would have the dependence inverted. The lag enters only as $\sqrt{D}$, far milder than the exponential decay of the ceiling in (4). And the hardest level to sit at is intermediate rather than extreme, which is the qualitative reason a controller gains from measuring $r(D)$ rather than committing to one branch in advance. Deriving the branch value curves, and with them a performance theorem for Algorithm 1, is left open.

Equation (5) is exact as written. The convenient reading $s_\theta \sigma_\theta \approx \theta \sqrt{2D \log(1/\theta)}$ holds only for large D at fixed θ and

fails at small D , since at $D = 1$ with $\theta \rightarrow 0$ the left side tends to 1 while that expression tends to 0.

Scope. Three assumptions earn the closed form. First, the contended unit is binary and symmetric. For any symmetric stationary x_t the value $\frac{1}{2}r(D)$ is achievable by the memoryless threshold $m_t = \hat{x}_t$, and it is the exact advantage when x_t is Markov, since then $\hat{x}_t$ is a sufficient statistic. For a general non-Markov symmetric process a history-using controller can do strictly better, as a random-phase square wave with $r(D)=0$ stays exactly predictable from its phase, so the equality is Markov-specific and the general converse is the mixing bound of App. A. Second, stationarity, since $r(D)$ is a lag- D autocorrelation. Under slow drift it is replaced by a local estimate, which is what Algorithm 1 tracks. Third, fixed D . If the freshest sample has a random age A the controller can read, the advantage is the age-averaged $\frac{1}{2} \mathbb{E}[r(A)]$ and the impossibility persists whenever $A \gg \tau_c$ almost surely.

Integer capacity beyond a single contended unit is left open. A layer-cake decomposition of $\min(w, a)$ and $(w - a)^+$ suggests one binary decision per unit, but the per-layer decisions must be kept nested to correspond to a scalar window, the layerwise advantage is only an upper bound unless a monotonicity condition holds, and scoring surplus by amount rather than by violation probability changes the problem. We prefer to state the single-unit case exactly than the general case loosely.

VI. REALIZATION: SENSING OVER A FEDERATED FABRIC

The controller runs inside the cluster, behind the network boundary, and reaches the home node over a NATS [19] leaf link that needs one outbound port (Fig. 1). It probes cluster occupancy and actuates admission across that link. Submissions flow on `burst.submit` and results return on `results.>` by interest propagation, with no inbound cluster access. The capacity feedback itself is local, since the controller and the workers both sit inside the cluster, so the leaf link carries submissions and results rather than the capacity signal. What matters for the controller is that its view of capacity is late for the reasons of (2). The leaf round trip and the CUDA start-up of a freshly admitted pod together set the information-to-effect lag of §IV, which we measure in §VIII. Tensors crossing the link are compressed by random-rotation quantization [20], [21], which helps bandwidth on the hop but does not reduce that lag, since it is dominated by CUDA start-up rather than transport. The fabric’s own design, its leaf federation, durability and single-port traversal, is characterized in the companion paper [2].

VII. IMPLEMENTATION

PoliteBurst is a Go control plane with a decider, prober, submitter, NATS bridge and back-off modules, plus a Python client and an IPython cell magic [22]. The controller runs inside the cluster under a minimal namespace-scoped ServiceAccount holding Job create, get and delete rights and pod read, connects to the in-cluster NATS leaf, and renders each `JobDescriptor` into a Kubernetes Job. Node targeting

through a `nodeSelector` on the descriptor pins guest pods to non-reserved GPU types. We verified the full path live on the reference host [1]. A submit from the home node propagated through hub, leaf and controller, launched a GPU worker that ran real CUDA work, and published its result back through the leaf to the home driver. The system, the controller and worker images, and the transport and right-sizing packages [20], [23] are available as open-source artifacts, with repositories anonymized for review.

VIII. EVALUATION

We evaluate on a testbed whose information-to-effect lag is measured rather than assumed. A home node and a workstation-class node with GV100 GPUs run the controller, which probes real GPU occupancy and admits real CUDA workers while a competing tenant drives capacity. The fabric carrying this control plane is characterized in the companion paper [2]. Unless noted we report the mean and a t -based 95% confidence interval over independent repeats. Four questions organize the study, and each one defends a claim the theory makes. **extbf(Q1)** What is the lag D of (2) that governs Prop. `efprop:bound`, and how is it composed. **(Q2)** Does Algorithm 1 stay polite while beating a fixed budget under real time-varying contention, and where does it fail. **(Q3)** Does the value $\frac{1}{2}r(D)$ of Thm. 1 hold at the controller level, and does it also bind a learned controller.

A. The lag and the violation bound (Q1)

The lag has two measured parts. A read-only `nvidia-smi` probe costs 27 to 33 ms at the median. The end-to-end delay from launching a real GPU load until it is visible as utilization above the 40% floor is **2.46 s** over 10 thermally guarded repeats on a GV100, with median 2.48 and range 2.29 to 2.61. CUDA start-up dominates that figure, not the poll. Which value applies depends on the question. Detecting whether a newly admitted pod is productive pays the full start-up cost, while probing pods that are already running costs only the poll. One epoch is a single completion-gated admission decision whose measured mean duration is about 1.1 s, so the 2.46 s start-up age is $D \approx 2$ to 3 epochs, which we sweep in Q3, and the 27 ms poll is below one epoch for running pods. Substituting into Prop. 2 makes D a first-order term in achievable politeness. A guest that admits aggressively with large α pays for it in over-admission scaled by this measured start-up delay.

B. Politeness under real contention (Q2)

We manufacture controlled contention on a two-GPU node with $C=2$. A competing tenant occupies $c(t) \in \{0, 1\}$ GPUs on a fixed profile, so the guest’s available capacity $a(t) = C - c(t)$ is exogenous, time-varying and measured by the prober through a per-GPU process census subject to the lag of Q1. Two profiles bracket the space. *Square* load is a period- 2τ wave modelling slowly varying or scheduled neighbours, and *poisson* load is memoryless on and off arrivals, the adversarial case. Alongside goodput and ρ we

TABLE I

POLITENESS UNDER REAL CONTENTION WITH $C=2, 4$ SEEDS, MEAN WITH 95% INTERVAL. “KEPT” IS COMPETITOR THROUGHPUT RETAINED, WHERE 1 IS UNDISTURBED. UNDER SQUARE LOAD ALGORITHM 1 IS THE POLITE AND EFFICIENT KNEE, AND UNDER POISSON LOAD FEEDBACK BREAKS EVEN, A PRE-REGISTERED NULL.

contention	policy	goodput (tasks/s)	ρ	kept
square	naive	0.101 ± 0.002	0.64 ± 0.01	0.62
	aimd	0.091 ± 0.000	0.15 ± 0.01	0.76
	static	0.064 ± 0.000	0.11 ± 0.01	0.89
poisson	naive	0.096 ± 0.002	0.85 ± 0.06	0.62
	aimd	0.062 ± 0.006	0.30 ± 0.06	0.55
	static	0.064 ± 0.000	0.22 ± 0.04	0.79

measure harm to the neighbour as the competitor’s completed work relative to its undisturbed baseline. The study was pre-registered with randomized interleaved order and four seeds per cell. Parameters, fixed before the runs, are $K=4$, $w_{\min}=1$, $\alpha=0.5$, $\beta=0.5$, $\theta=0.15$, $\bar{a}=1$, a 60 s square period, a Poisson rate matched to the same duty cycle, an 8 s GPU work unit, and $B=64$ submitted jobs per trial, with each trial ending when the burst drains.

Table I reports it. Under structured load Algorithm 1 is the polite and efficient knee. It reaches 1.41 times a fixed rate’s goodput, 0.091 against 0.064 tasks/s with disjoint intervals, at $\rho = 0.15$, which is a quarter of naive’s 0.64, and it spares the neighbour, retaining 76% of its throughput against naive’s 62%. The target ϵ and switch level θ were fixed before the runs. Host policy penalizes persistently under-utilized requests rather than momentary dips, so ϵ between 0.1 and 0.2 is the tolerable share of under-utilized epochs, and the measured 0.15 sits inside it. Naive’s higher goodput of 0.101 is bought entirely at the neighbour’s expense, since it over-admits two-thirds of the time and takes 38% of the competitor’s throughput.

Under memoryless load feedback breaks even and we report it. Goodput of 0.062 is statistically indistinguishable from the fixed rate’s 0.064 with overlapping intervals, and over-admission is worse at $\rho = 0.30$, because capacity now varies on the timescale of the sensing delay. By the time the prober sees a competitor arrive the observation is stale. This is a pre-registered null and it is what Thm. 1 predicts, since the two profiles are two coherence times on the same curve (Cor. 2). Slow square load has $\tau_c \gg D$ and gives feedback room to help, while memoryless load drives $r(D)$ toward zero, where no causal controller beats a fixed budget. The break-even is therefore a consequence of the theorem rather than a tuning failure, and it is the case Algorithm 1 is built to detect. The measured sensing path also places the Prop. 2 sawtooth ceiling above the observed ρ in both profiles, which is conservative for a discrete completion-gated controller, and its prediction that politeness degrades as contention outpaces D is borne out. Naive is the most impolite in every case, at ρ between 0.64 and 0.85.

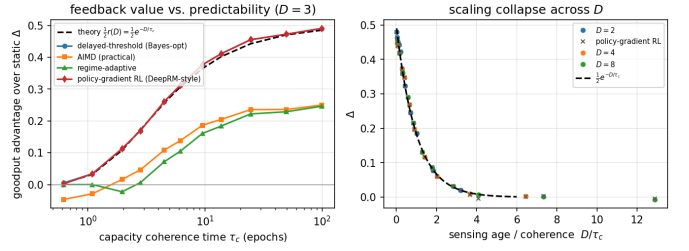


Fig. 3. Controller-level validation. *Left*. Goodput advantage over a fixed budget against capacity coherence time τ_c at $D=3$. The threshold policy lies on the predicted $\frac{1}{2}r(D)$, dashed, to within 0.013, and the DeepRM-style policy-gradient agent, diamonds, converges onto it without passing it. Fixed AIMD falls below a fixed budget once $\tau_c \lesssim D$, and Algorithm 1 tracks the envelope. *Right*. Sweeping D collapses the advantage onto $\frac{1}{2}e^{-D/\tau_c}$ against D/τ_c .

C. The value of feedback at the controller level (Q3)

The two contention profiles are two points, while Thm. 1 predicts the whole curve. To test it we simulate the window controllers on the two-state capacity process the theory analyzes, a Markov competitor with per-epoch flip probability p so the coherence time τ_c sweeps continuously, at the measured lag $D=3$ epochs with 1.2×10^5 epochs per point. Fig. 3 reports each controller’s goodput advantage over the best fixed budget at matched over-admission against the predicted $\frac{1}{2}r(D)$.

Four readings confirm the theory and motivate Algorithm 1. First, the threshold policy sits on the predicted line across three decades of τ_c , with maximum deviation 0.013, so the closed form is the exact frontier rather than an asymptote. Second, fixed AIMD captures roughly half the available advantage when $\tau_c \gg D$ and goes negative, meaning worse than a fixed budget, once $\tau_c \lesssim D$, because it keeps paying over-admission for a signal that has become noise. That is the same failure the live poisson case shows. Third, Algorithm 1, estimating $r(D)$ online, tracks the upper envelope. It never drops below a fixed budget and recovers most of AIMD’s advantage where feedback is worth having. Fourth, the value binds a learned controller. A policy-gradient agent in the DeepRM family [24], using a tabular softmax with a per-state baseline [25] and trained on a history of four delayed readings rather than the newest one alone, converges to the threshold policy’s advantage within 0.003 at every point and exceeds $\frac{1}{2}r(D)$ by at most 0.013, the threshold policy’s own sampling deviation. Learning finds the frontier and does not pass it, which is the ceiling Thm. 1 sets for that family [24], [26]. Under the Markov model the extra readings are conditionally redundant given the newest one, so the experiment tests whether a learner recovers the sufficient statistic rather than whether extra history helps. Sweeping D collapses every curve onto $\frac{1}{2}e^{-D/\tau_c}$ against D/τ_c (Fig. 3, right), confirming the predicted scaling. The live square and poisson cases of Q2 are the two ends of that axis.

Live behaviour of the switch. Running Algorithm 1 on the same two-GPU node across the Markov sweep (p from 0.01 to 0.4, two seeds) confirms two mechanical claims. The online

estimate tracks the model, with $\hat{r} = (1 - 2\hat{p})^D$ within 0.07 of $(1 - 2p)^D$, and the switch modulates as intended, its closed-loop fraction falling from 0.98 to 0.48 as $r(D)$ drops past θ . It does not reproduce the goodput crossover. Raw goodput rewards over-admission, so live AIMD out-performs a fixed budget everywhere, buying that goodput with impoliteness at high flip rate ($\rho = 0.44$ against 0.04) rather than by tracking. What the switch delivers on hardware is the over-admission it sheds against always running AIMD, growing from +0.01 to +0.22 as p rises from 0.08 to 0.4. Two seeds make this a mechanical check on the estimator and the switch, not evidence for the value of $\Delta(D)$.

D. Summary and threats to validity

The evaluation establishes three things. The lag is a measured first-order term at about 2.5s, dominated by worker start-up rather than by polling, which is the composition (2) assumes. Under structured contention Algorithm 1 is the polite and efficient knee and spares the neighbour, while under memoryless contention feedback breaks even, a pre-registered null we report rather than suppress. And the value $\frac{1}{2}r(D)$ holds at the controller level, with the threshold policy on it to within 0.013, a learned policy-gradient agent capped by it, and Algorithm 1 never below a fixed budget, which unifies the two live contention cases as endpoints of one curve.

Three threats remain, and the first is a limit on what the live numbers can show. Thm. 1 compares feedback against the best observation-oblivious policy *at matched over-admission*, whereas the live fixed policy is a single trickle with no tunable frontier. The $1.41\times$ figure is therefore a systems measurement of one operating point and *not* evidence for the value $\frac{1}{2}r(D)$. Only the controller-level sweep of §VIII-C tests that value, and it does so on the process the theorem assumes. A live matched- ρ frontier, sweeping randomized fixed budgets and feedback parameters and comparing maximum goodput at equal ρ , is the experiment that would close this and we have not run it. Second, contention is manufactured on a two-GPU node at $C=2$ with synthetic profiles rather than drawn from a production multi-tenant trace. Third, repetition is thin at four seeds for Table I and two for the live sweep, the “kept” column carries no interval, and intervals printed as 0.000 are below the resolution shown rather than exactly zero. The full system path is nonetheless verified end-to-end in §VII, and naive’s impoliteness, at $\rho \geq 0.64$ with at least 38% neighbour harm, is robust across every case.

IX. RELATED WORK

Bursting and function execution. HTCondor [3], funcX and Globus Compute [4] and Ray [5] provide elastic task execution and provider-driven scaling, and treat the host as owned capacity to fill. None of them models a fair-use politeness budget or admission under a delayed view of capacity. They are complementary substrates, and a head-to-head comparison against them appears in the companion systems paper [2].

Congestion and scavenger control. Our loop is AIMD in the lineage of additive-increase and multiplicative-decrease

congestion avoidance [27], and its objective of yielding to incumbents and using only slack aligns with low-priority transports such as LEDBAT [28] and TCP Nice [29]. Those protocols back off on one-sided loss or delay signals. Two things differ here. The objective is two-sided, since the guest must stay under a pod cap and above a utilization floor, which admits an over-admission measure rather than pure loss. And we bound what any such rule can achieve rather than proposing one more back-off schedule.

Opportunistic and harvesting schedulers. Running low-priority work in another tenant’s slack is well established. AntMan co-schedules opportunistic deep-learning jobs against production ones on shared GPUs [30], Harvest VMs sell reclaimable idle cloud capacity with an SLO for the primary [31], and Gavel [32] and Pollux [33] adapt GPU allocations dynamically. All of them are operated by the party that owns the capacity, so available capacity is observable and the constraint is one-sided, namely do not harm the primary. The guest here owns nothing, cannot see the neighbour directly, and must also stay *above* a floor on what it does hold.

Learned resource control. Deep reinforcement learning schedulers including DeepRM [24] and Decima [26], with variance reduction for exogenous input processes [25], learn scheduling policies from cluster feedback. Our ceiling applies to them unchanged, since it constrains any causal policy rather than any particular family, and §VIII-C tests a policy-gradient agent of that family against it.

Age of information. The ceiling is a statement about freshness, since the value of feedback is set by the age of the capacity estimate relative to how fast the neighbour changes. That places polite bursting in the age-of-information line of work [34], on the control side rather than the sampling side. We bound the achievable goodput and politeness frontier as a function of that lag and give a controller that adapts to the measured value.

X. CONCLUSION

Restraint on a shared cluster is a control problem whose difficulty is fixed by how far its decisions land behind the state they are judged against. We gave that dependence an exact form. The extra goodput a feedback controller can win over the best fixed budget, compared at equal over-admission, is half the correlation between capacity when the decision lands and capacity as last seen. It is attained by admitting work whenever the last reading showed capacity free, and it falls toward zero once that lag outgrows the neighbour’s correlation time.

One controller follows from that form. It measures the correlation online and switches between a feedback loop and a fixed budget, trailing the better of the two on average by an amount that shrinks with run length, and it never exceeds the guest’s own window, which holds deterministically rather than in expectation. We also bound from both sides how hard it is to decide which branch to run, and that cost grows only with the square root of the lag even though the ceiling it chases decays exponentially in the same quantity. Converting that decision

cost into a goodput guarantee needs the value of each branch under a Markov neighbour, which we leave open.

Short controlled experiments support the result on both sides of the crossover. Under structured neighbour load, feedback admission reaches 1.41 times a fixed budget's goodput at a quarter of a greedy policy's over-admission while leaving the competing tenant more throughput. Under memoryless load it breaks even, and the switching algorithm detects the case and stops paying for a loop that cannot help. A learned policy-gradient controller given a longer observation history converges to the ceiling and does not pass it, on the process the theorem assumes.

Four things remain open. Contention is manufactured on a two-GPU node rather than drawn from a production multi-tenant trace, and the live comparison is at one operating point rather than a matched-over-admission frontier, so it measures the system rather than the constant. The exact equality holds for a binary Markov unit, while the general statement is the scalarized bound of App. A. Integer capacity needs the nesting and monotonicity conditions of §V before it can be claimed. And a controller that estimates the correlation itself, rather than only which side of the switch it falls on, should recover part of the distance between AIMD and the ceiling.

APPENDIX A

GENERAL-CAPACITY IMPOSSIBILITY VIA β -MIXING

Corollary 1 showed that feedback's value vanishes with the lag D for the two-state model. The same holds for any stationary capacity process, with the decay rate set by its β -mixing coefficient, so the exact binary law is one tight instance of a general statement.

Let $\{a_t\}$ be stationary on $\{0, \dots, C\}$ with $\mathcal{F}_{\leq s} = \sigma(a_u : u \leq s)$ and $\mathcal{F}_{\geq s} = \sigma(a_u : u \geq s)$, and let

$$\beta(D) = \mathbb{E} \left[\sup_{B \in \mathcal{F}_{\geq t}} |\mathbb{P}(B \mid \mathcal{F}_{\leq t-D}) - \mathbb{P}(B)| \right], \quad (7)$$

which tends to 0 exactly when the process is absolutely regular and satisfies $\beta(D) \leq c_0 \varrho^D$ when it mixes geometrically. Note $\beta(D)$ always carries its lag and is unrelated to the AIMD back-off β . Fix a bounded per-epoch objective $R(w, a)$ with oscillation $\Omega \triangleq \max_{w,a} R - \min_{w,a} R$, for instance the politeness Lagrangian $R_\lambda = \min(w, a) - \lambda(w - a)^+$. A causal lag- D controller makes w_t measurable with respect to $\mathcal{G} \triangleq \mathcal{F}_{\leq t-D}$ and a fixed one uses a $\mathcal{G}$ -independent w , with optimal per-epoch values $V^*(D)$ and V_{stat} and $\Delta(D) = V^*(D) - V_{\text{stat}}$.

Theorem 2 (β -mixing impossibility). *For any such process and objective, $0 \leq \Delta(D) \leq \Omega \beta(D)$. Hence no causal lag- D controller beats a fixed budget as $D \rightarrow \infty$ whenever the capacity is absolutely regular, and the advantage decays geometrically when the process does.*

Proof sketch. Write $\mu_{\mathcal{G}} = \mathbb{P}(a_t \in \cdot \mid \mathcal{G})$ and $\psi(\mu) = \max_w \sum_a R(w, a) \mu(a)$. Optimizing the $\mathcal{G}$ -measurable window pointwise gives $V^*(D) = \mathbb{E}[\psi(\mu_{\mathcal{G}})]$ and $V_{\text{stat}} = \psi(\bar{\mu})$, so $\Delta(D) \geq 0$ by convexity of ψ and Jensen, which is the statement that fresher information never hurts. For the upper

bound, ψ is a pointwise maximum of linear functionals and therefore Ω -Lipschitz in total variation, the standard value-of-information bound [12], [13]. Hence $\Delta(D) \leq \Omega \mathbb{E} \|\mu_{\mathcal{G}} - \bar{\mu}\|_{\text{TV}} \leq \Omega \beta(D)$, the last step because $\sigma(a_t) \subseteq \mathcal{F}_{\geq t}$, so (7) takes its supremum over a larger field. $\square$

For the symmetric two-state model $\beta(D) = \frac{1}{2}|1 - 2q(D)| = \frac{1}{2}r(D)$ by a standard mixing identity [14], [9], since the Markov property reduces $\mathcal{F}_{\leq t-D}$ to a_{t-D} and the conditional-to-marginal total variation is $|\frac{1}{2} - q(D)|$ from either state. So Thm. 2 recovers the binary rate up to the objective scale Ω . One limitation should be stated plainly. Thm. 2 bounds $V^*(D) - V_{\text{stat}}$ for a *scalar* objective such as a Lagrangian, whereas Thm. 1 is a statement about the constrained frontier at every matched ρ . Transferring a support-function bound to the constrained frontier needs a duality argument we do not give, so this appendix is a scalarized value-of-information result that agrees with the binary law rather than a generalization of it.

APPENDIX B

THE BRANCH-DECISION RATE

We give both directions of Lemma 2. Observations are delayed but clean, so $\hat{a}_i = a_{i-D}$, and Algorithm 1 forms $\hat{p}_n = \frac{1}{n} \sum_{i \leq n} \mathbf{1}[\hat{a}_i \neq \hat{a}_{i-1}]$ and $\hat{r}_n = (1 - 2\hat{p}_n)^D$.

Since $\Pr[a_t \neq a_{t-1} \mid a_0, \dots, a_{t-1}] = p$ for every history, $n\hat{p}_n$ is exactly Binomial(n, p), with variance $p(1-p)/n$ and per-sample Fisher information $1/(p(1-p))$. A generic reversible-chain Hoeffding bound would replace that variance by a range proxy of order $1/\gamma$, overstating the sample requirement by order $1/\gamma^2$ and inverting the sign of the γ dependence, so we use the Bernstein form of [18], which is stated in the asymptotic variance and is exact here.

Upper bound. Bernstein gives $|\hat{p}_n - p| = O(\sigma_\theta \sqrt{\log(1/\delta)/n})$ with probability $1 - \delta$. The map $p \mapsto (1 - 2p)^D$ has derivative s_θ at p_θ , so $|\hat{r}_n - r(D)| = O(s_\theta \sigma_\theta \sqrt{\log(1/\delta)/n})$. A wrong branch costs at most L times the shortfall in r , and summing while n grows, then dividing by the horizon, gives $O(L s_\theta \sigma_\theta / \sqrt{N})$.

Lower bound. Take two processes with flip probabilities $p_\theta \pm \delta/2$. Independence of the flip indicators makes the Kullback-Leibler divergence between n -sample laws $n\delta^2/(2\sigma_\theta^2) + O(n\delta^3)$, so no test separates them with constant probability unless $n \gtrsim \sigma_\theta^2/\delta^2$. Their correlations differ by $s_\theta \delta$, so a rule that cannot distinguish them errs by $\Omega(s_\theta \delta)$ in r on one of them. Le Cam's two-point argument [11] with $\delta = \sigma_\theta/\sqrt{N}$ matches the upper bound. The two flip probabilities differ by $O(N^{-1/2})$, so their spectral gaps agree to that order and the pair certifies the rate without invoking a class boundary. That matters because θ is fixed by the objective, so on the manifold $r(D) = \theta$ the quantities D , $\gamma_\theta = 1 - \theta^{1/D}$ and s_θ are mutually determined, and treating them as free is what yields a spurious $1/\sqrt{\gamma_0 N}$ rate.

Worst switch level. With $u = 1 - 2p$ and $\theta = u^D$ we have $s_\theta \sigma_\theta = D u^{D-1} \sqrt{1 - u^2}$. Setting the derivative of $(D-1) \log u + \frac{1}{2} \log(1 - u^2)$ to zero gives $u^2 = (D-1)/D$, so $\sqrt{1 - u^2} = 1/\sqrt{D}$ and $u^{D-1} \rightarrow e^{-1/2}$. The product is $e^{-1/2} \sqrt{D}$ at $\theta \rightarrow e^{-1/2}$, which is (6).

REFERENCES

- [1] D. Weitzel, A. Graves, S. Albin, H. Zhu, F. Würthwein, M. Tatineni, D. Mishin, J. Graham, E. E. Khoda, M. F. Sada, L. Smarr, and T. DeFanti, “The national research platform: Stretched, multi-tenant, scientific Kubernetes cluster,” *arXiv preprint arXiv:2505.22864*, 2025.
- [2] Anonymous Author(s), “A policy-aware, container-native bursting fabric with federated messaging, transport compression, and thermal-aware right-sizing,” Companion systems paper; anonymized for double-blind review, available to reviewers on request.
- [3] D. Thain, T. Tannenbaum, and M. Livny, “Distributed computing in practice: the Condor experience,” *Concurrency and Computation: Practice and Experience*, vol. 17, no. 2-4, pp. 323–356, 2005.
- [4] R. Chard, Y. Babuji, Z. Li, T. Skluzacek, A. Woodard, B. Blaiszik, I. Foster, and K. Chard, “funcX: A federated function serving fabric for science,” in *Proc. 29th Int. Symp. on High-Performance Parallel and Distributed Computing (HPDC)*, 2020.
- [5] P. Moritz, R. Nishihara, S. Wang *et al.*, “Ray: A distributed framework for emerging AI applications,” in *Proc. 13th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 2018.
- [6] K. Ousterhout, P. Wendell, M. Zaharia, and I. Stoica, “Sparrow: Distributed, low latency scheduling,” in *Proc. 24th ACM Symp. on Operating Systems Principles (SOSP)*, 2013.
- [7] M. Mitzenmacher, “How useful is old information?” *IEEE Transactions on Parallel and Distributed Systems*, vol. 11, no. 1, pp. 6–20, 2000.
- [8] A. Goel, A. Meyerson, and S. Plotkin, “Distributed admission control, scheduling, and routing with stale information,” in *Proc. 12th ACM-SIAM Symp. on Discrete Algorithms (SODA)*, 2001, pp. 611–619.
- [9] D. A. Levin, Y. Peres, and E. L. Wilmer, *Markov Chains and Mixing Times*, 2nd ed. American Mathematical Society, 2017.
- [10] S. H. A. Ahmad, M. Liu, T. Javidi, Q. Zhao, and B. Krishnamachari, “Optimality of myopic sensing in multichannel opportunistic access,” *IEEE Transactions on Information Theory*, vol. 55, no. 9, pp. 4040–4050, 2009.
- [11] A. B. Tsybakov, *Introduction to Nonparametric Estimation*. Springer, 2009.
- [12] R. A. Howard, “Information value theory,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 2, no. 1, pp. 22–26, 1966.
- [13] M. De Lara and O. Gossner, “Payoffs-beliefs duality and the value of information,” *SIAM Journal on Optimization*, vol. 30, no. 1, pp. 464–489, 2020.
- [14] R. C. Bradley, “Basic properties of strong mixing conditions: A survey and some open questions,” *Probability Surveys*, vol. 2, pp. 107–144, 2005.
- [15] S.-H. Kim, “State information lag markov decision process with control limit rule,” *Naval Research Logistics Quarterly*, vol. 32, pp. 491–496, 1985.
- [16] D. M. Brooks and C. T. Leondes, “Markov decision processes with state-information lag,” *Operations Research*, vol. 20, no. 4, pp. 904–907, 1972.
- [17] K. V. Katsikopoulos and S. E. Engelbrecht, “Markov decision processes with delays and asynchronous cost collection,” *IEEE Transactions on Automatic Control*, vol. 48, no. 4, pp. 568–574, 2003.
- [18] D. Paulin, “Concentration inequalities for Markov chains by Marton couplings and spectral methods,” *Electronic Journal of Probability*, vol. 20, no. 79, pp. 1–32, 2015.
- [19] “NATS: Connective technology for adaptive edge and distributed systems,” <https://nats.io>, accessed 2026.
- [20] Anonymous Author(s), “A random-rotation quantization library for embedding and KV-cache compression,” Software artifact; repository anonymized for double-blind review, available to reviewers on request.
- [21] A. Zandieh, M. Daliri, M. Hadian, and V. Mirrokni, “TurboQuant: Online vector quantization with near-optimal distortion rate,” *arXiv preprint arXiv:2504.19874*, 2025.
- [22] Anonymous Author(s), “The bursting fabric described in this paper,” Software artifact; repository anonymized for double-blind review, available to reviewers on request.
- [23] —, “An OOM-safe GPU batch-sizing and thermal-aware control library,” Software artifact; repository anonymized for double-blind review, available to reviewers on request.
- [24] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, “Resource management with deep reinforcement learning,” in *Proc. 15th ACM Workshop on Hot Topics in Networks (HotNets)*, 2016.
- [25] H. Mao, S. B. Venkatakrishnan, M. Schwarzkopf, and M. Alizadeh, “Variance reduction for reinforcement learning in input-driven environments,” in *Proc. 7th Int. Conf. on Learning Representations (ICLR)*, 2019.
- [26] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, “Learning scheduling algorithms for data processing clusters,” in *Proc. ACM SIGCOMM*, 2019.
- [27] D.-M. Chiu and R. Jain, “Analysis of the increase and decrease algorithms for congestion avoidance in computer networks,” *Computer Networks and ISDN Systems*, vol. 17, no. 1, pp. 1–14, 1989.
- [28] S. Shalunov, G. Hazel, J. Iyengar, and M. Kuehlewind, “Low extra delay background transport (LEDBAT),” RFC 6817, IETF, 2012.
- [29] A. Venkataramani, R. Kokku, and M. Dahlin, “TCP Nice: A mechanism for background transfers,” in *Proc. 5th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 2002.
- [30] W. Xiao, S. Ren, Y. Li, Y. Zhang, P. Hou, Z. Li, Y. Feng, W. Lin, and Y. Jia, “Antman: Dynamic scaling on GPU clusters for deep learning,” in *Proc. 14th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 2020.
- [31] P. Ambati, N. Bashir, D. Irwin, and P. Shenoy, “Providing slos for resource-harvesting vms in cloud platforms,” in *Proc. 14th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 2020.
- [32] D. Narayanan, K. Santhanam, F. Kazhamiaka, A. Phanishayee, and M. Zaharia, “Heterogeneity-aware cluster scheduling policies for deep learning workloads,” in *Proc. 14th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 2020.
- [33] A. Qiao, S. K. Choe, S. J. Subramanya, W. Neiswanger, Q. Ho, H. Zhang, G. R. Ganger, and E. P. Xing, “Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning,” in *Proc. 15th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 2021.
- [34] R. D. Yates, Y. Sun, D. R. Brown, S. K. Kaul, E. Modiano, and S. Ulukus, “Age of information: An introduction and survey,” *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 5, pp. 1183–1210, 2021.